

**LAPORAN PRAKTIKUM**  
**TEKNOLOGI IOT II**



Oleh :

Nama : LILI CAHYA PUTRI

NPM : 25782071

Kelas : TRI 3C

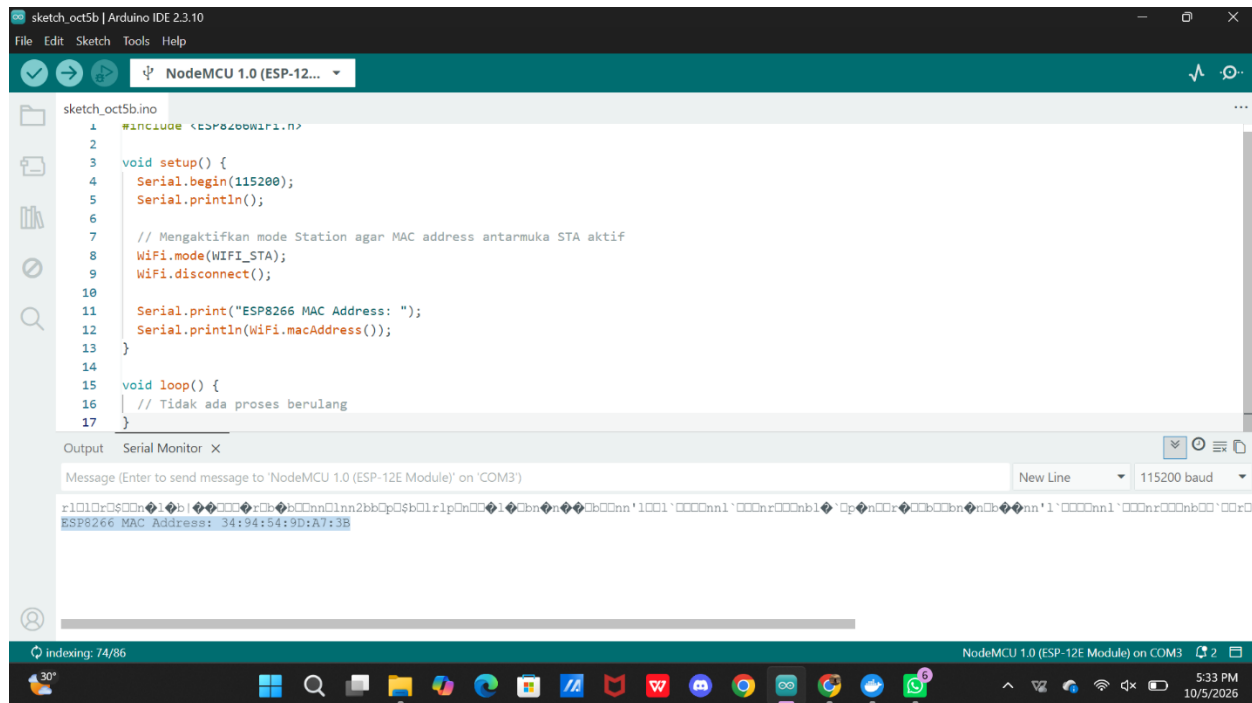
**PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET**  
**JURUSAN TEKNOLOGI INFORMASI**  
**POLITEKNIK NEGERI LAMPUNG**

**2026**

## Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)

### 5.5 Praktikum

#### 5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU



The screenshot shows the Arduino IDE interface. The top toolbar includes icons for File, Edit, Sketch, Tools, and Help. The board selector is set to 'NodeMCU 1.0 (ESP-12E Module)'. The sketch file is named 'sketch\_oct5b.ino'. The code in the editor is as follows:

```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

The Serial Monitor at the bottom shows the output: 'ESP8266 MAC Address: 34:94:54:9D:A7:3B'. The baud rate is set to 115200. The Windows taskbar at the bottom shows the time as 5:33 PM on 10/5/2026.

Berdasarkan pengujian pada Praktikum 5.5.1, dilakukan pembacaan identitas fisik (*MAC Address*) pada setiap modul NodeMCU ESP8266 menggunakan mode *Station* (*WIFI\_STA*). Pemanggilan fungsi `WiFi.macAddress()` berhasil menampilkan alamat *hardware* 48-bit yang unik secara global pada Serial Monitor (*baud rate* 115200).

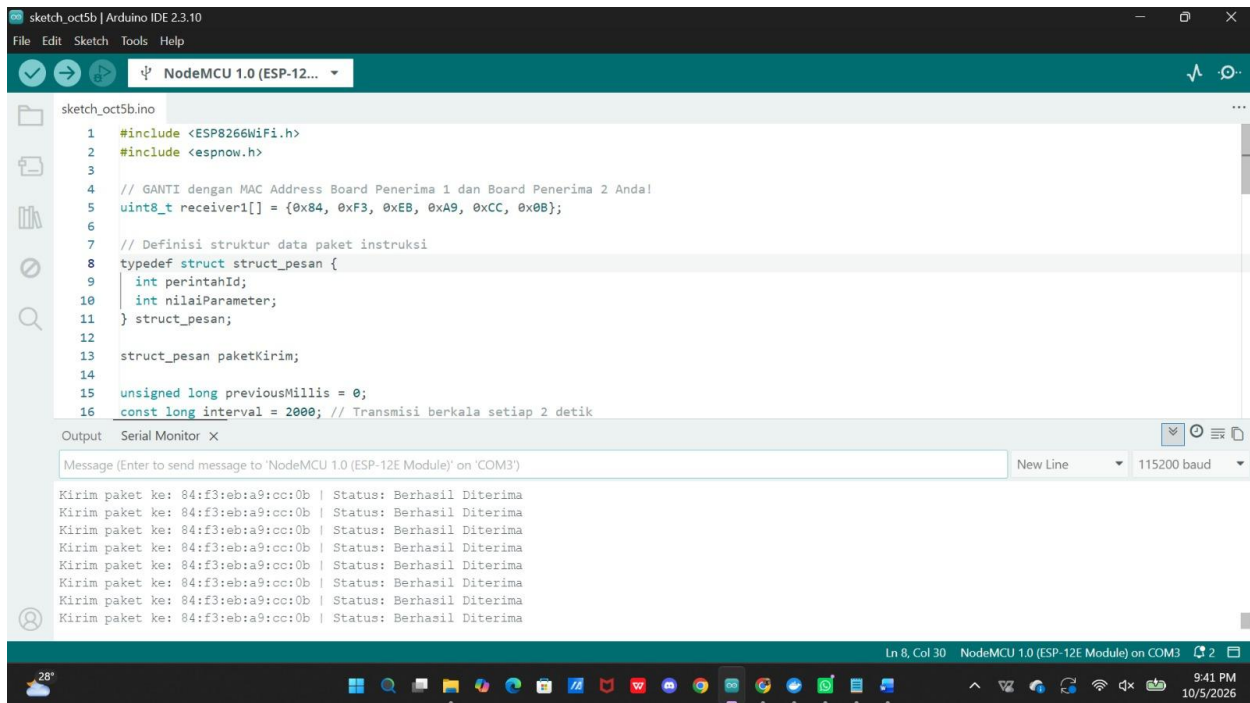
Dari hasil identifikasi, diperoleh *MAC Address* untuk masing-masing perangkat sebagai berikut:

- Node 1 (Controller / Pengirim): 34:94:54:9D:A7:3B
- Node 2 (Slave / Penerima): 84:F3:EB:A9:CC:0B

Pengidentifikasian dan pemberian label fisik pada setiap *board* ini krusial dilakukan sebelum melangkah ke tahap komunikasi ESP-NOW agar pendaftaran *peer* pada memori *routing* pengirim tepat target dan tidak tertukar.

## 5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

### Node 1 (Pengirim):



The screenshot shows the Arduino IDE interface for Node 1 (Sender). The code is in sketch\_oct5b.ino and includes ESP8266WiFi.h and espnow.h. It defines a receiver MAC address and a packet structure. The serial monitor shows the output of the program, which is sending packets to the receiver.

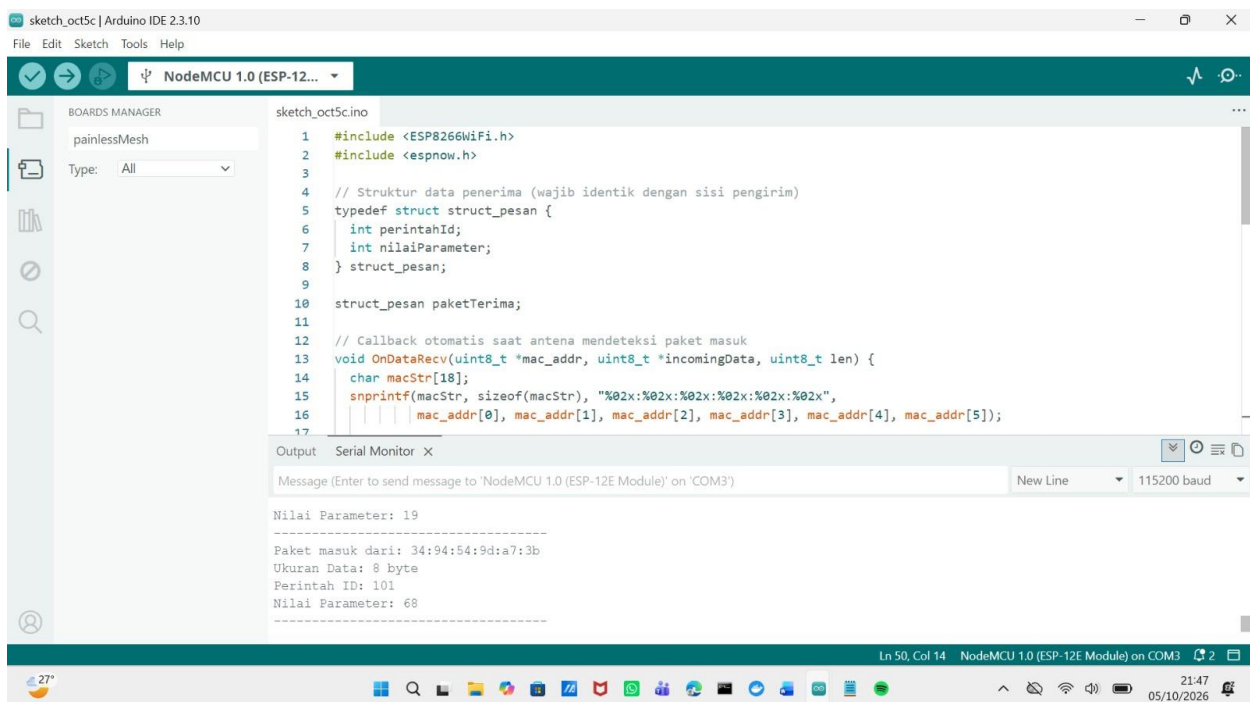
```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Anda!
5 uint8_t receiver[] = {0x84, 0xF3, 0xEB, 0xA9, 0xCC, 0x0B};
6
7 // Definisi struktur data paket instruksi
8 typedef struct struct_pesan {
9     int perintahId;
10    int nilaiParameter;
11 } struct_pesan;
12
13 struct_pesan paketKirim;
14
15 unsigned long previousMillis = 0;
16 const long interval = 2000; // Transmisi berkala setiap 2 detik
```

Output Serial Monitor X

```
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line 115200 baud

Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Berhasil Diterima
```

### Node 2 (Penerima):



The screenshot shows the Arduino IDE interface for Node 2 (Receiver). The code is in sketch\_oct5c.ino and includes ESP8266WiFi.h and espnow.h. It defines a packet structure and a callback function to handle incoming data. The serial monitor shows the output of the program, which is receiving packets from the sender.

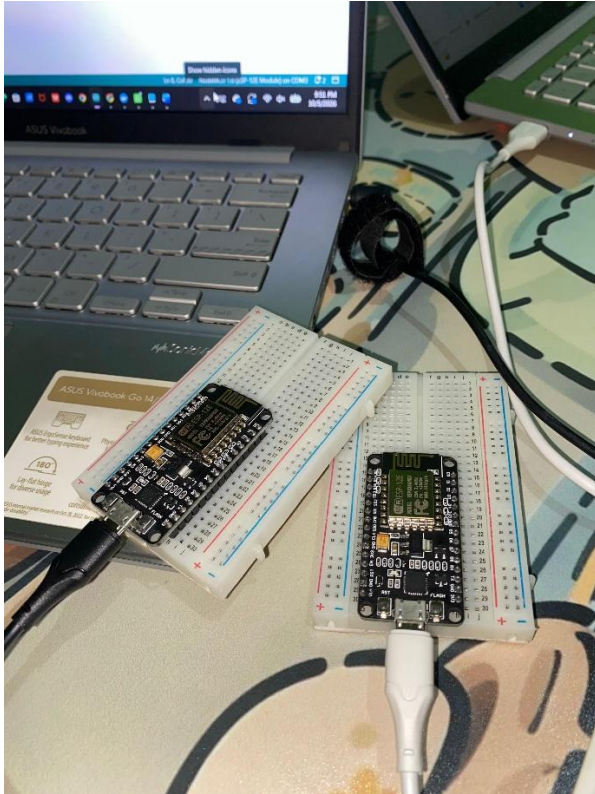
```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6     int perintahId;
7     int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     sprintf(macStr, "%02x:%02x:%02x:%02x:%02x:%02x",
16         mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17 }
```

Output Serial Monitor X

```
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
New Line 115200 baud

Nilai Parameter: 19
-----
Paket masuk dari: 34:94:54:9d:a7:3b
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 68
-----
```

## Dokumentasi:

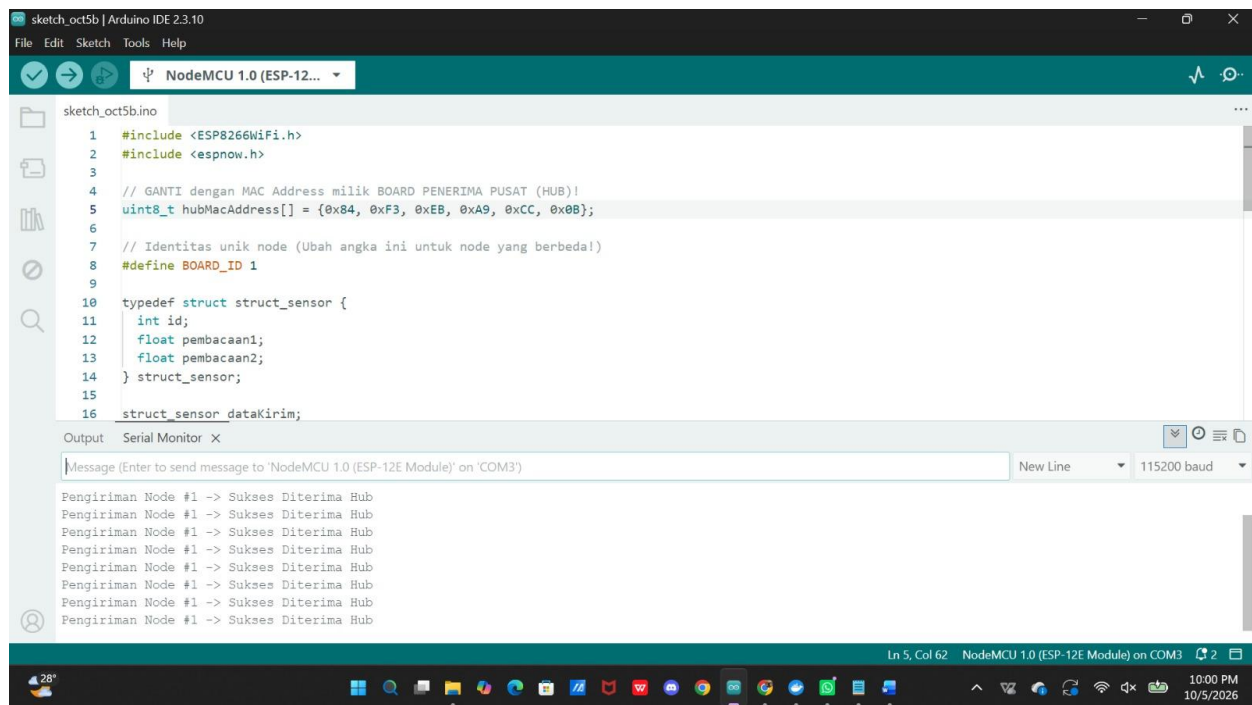


Pada Praktikum 5.5.2, dilakukan pengujian komunikasi *One-to-Many* menggunakan protokol ESP-NOW. Node 1 difungsikan sebagai *Controller* (pengirim) yang memancarkan paket instruksi terstruktur (`struct_pesan`) berisi `perintahId` dan `nilaiParameter` acak secara berkala setiap 2 detik.

Karena praktikum disesuaikan menggunakan 2 unit NodeMCU, Node 1 mendaftarkan *MAC Address* milik Node 2 (84:F3:EB:A9:CC:0B) ke dalam tabel *peer* menggunakan `esp_now_add_peer()`. Node 2 bertindak sebagai *Slave (Receiver)* yang mendengarkan dan menerima paket data tanpa perlu mengetahui *MAC Address* pengirim. Melalui fungsi *callback* interupsi `OnDataRecv()`, data mentah disalin ke variabel `paketTerima` dengan `memcpy()` dan ditampilkan secara *real-time* pada Serial Monitor *baud rate* 115200 dengan status pengiriman terkonfirmasi berhasil.

### 5.5.3 Praktikum 3: Pengumpulan Data Telemetry Many-to-One (ESP-NOW)

#### Node 1 (Pengirim):



The screenshot shows the Arduino IDE interface for Node 1. The code in `sketch_oct5b.ino` includes `ESP8266WiFi.h` and `espnw.h`. It defines a MAC address for the hub, a unique node ID (1), and a sensor structure. The `void setup()` function initializes the WiFi and prints the node ID. The `void loop()` function sends the sensor data to the hub. The serial monitor shows the output: "Pengiriman Node #1 -> Sukses Diterima Hub" repeated multiple times.

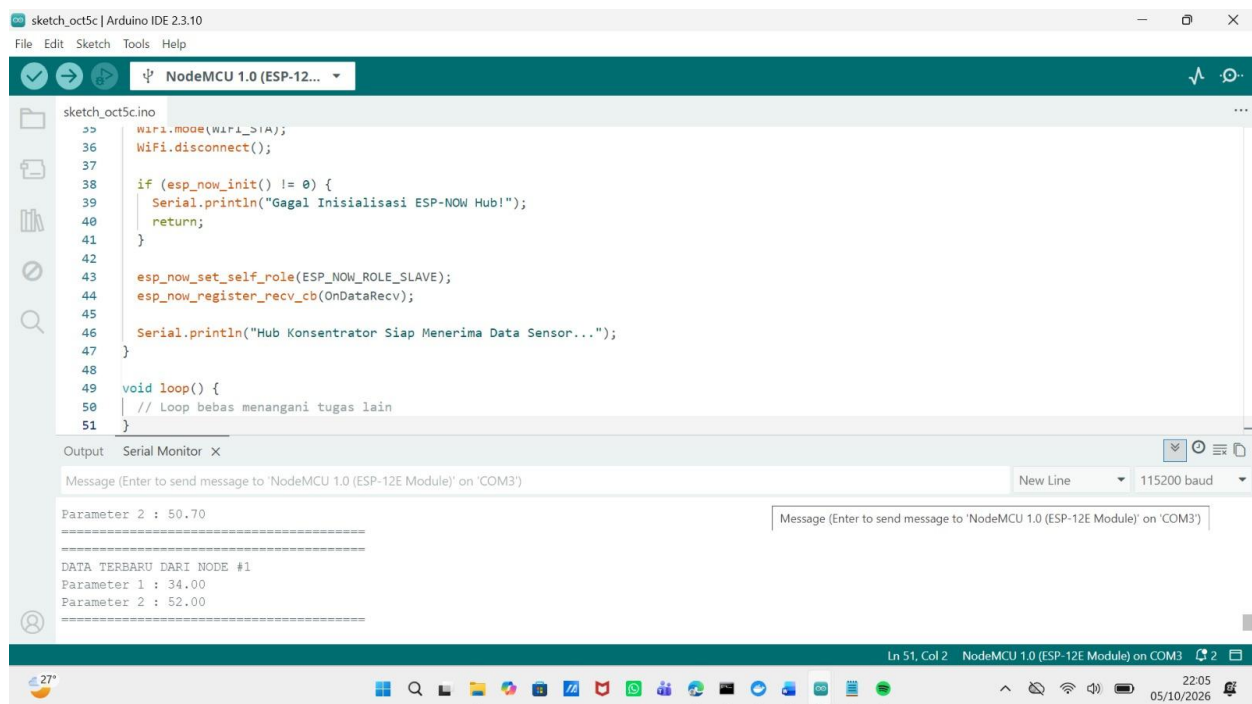
```
1 #include <ESP8266WiFi.h>
2 #include <espnw.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5 uint8_t hubMacAddress[] = {0x84, 0xF3, 0xEB, 0xA9, 0xCC, 0x08};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda!)
8 #define BOARD_ID 1
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Pengiriman Node #1 -> Sukses Diterima Hub  
Pengiriman Node #1 -> Sukses Diterima Hub  
Pengiriman Node #1 -> Sukses Diterima Hub  
Pengiriman Node #1 -> Sukses Diterima Hub  
Pengiriman Node #1 -> Sukses Diterima Hub  
Pengiriman Node #1 -> Sukses Diterima Hub  
Pengiriman Node #1 -> Sukses Diterima Hub  
Pengiriman Node #1 -> Sukses Diterima Hub

#### Node 2 (Penerima):



The screenshot shows the Arduino IDE interface for Node 2. The code in `sketch_oct5c.ino` includes `WiFi.h` and `espnw.h`. It defines a MAC address for the hub, a unique node ID (2), and a sensor structure. The `void setup()` function initializes the WiFi and prints the node ID. The `void loop()` function receives the sensor data from the hub and prints it. The serial monitor shows the output: "Parameter 2 : 50.70", "DATA TERBARU DARI NODE #1", "Parameter 1 : 34.00", "Parameter 2 : 52.00".

```
35 //wifi.mode(WIFI_STA);
36 WiFi.disconnect();
37
38 if (espnw_init() != 0) {
39     Serial.println("Gagal Inisialisasi ESP-NOW Hub!");
40     return;
41 }
42
43 espnw_set_self_role(ESP_NOW_ROLE_SLAVE);
44 espnw_register_rcv_cb(OnDataRecv);
45
46 Serial.println("Hub Konsentrator Siap Menerima Data Sensor...");
47 }
48
49 void loop() {
50     // Loop bebas menangani tugas lain
51 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Parameter 2 : 50.70  
=====

DATA TERBARU DARI NODE #1  
Parameter 1 : 34.00  
Parameter 2 : 52.00  
=====

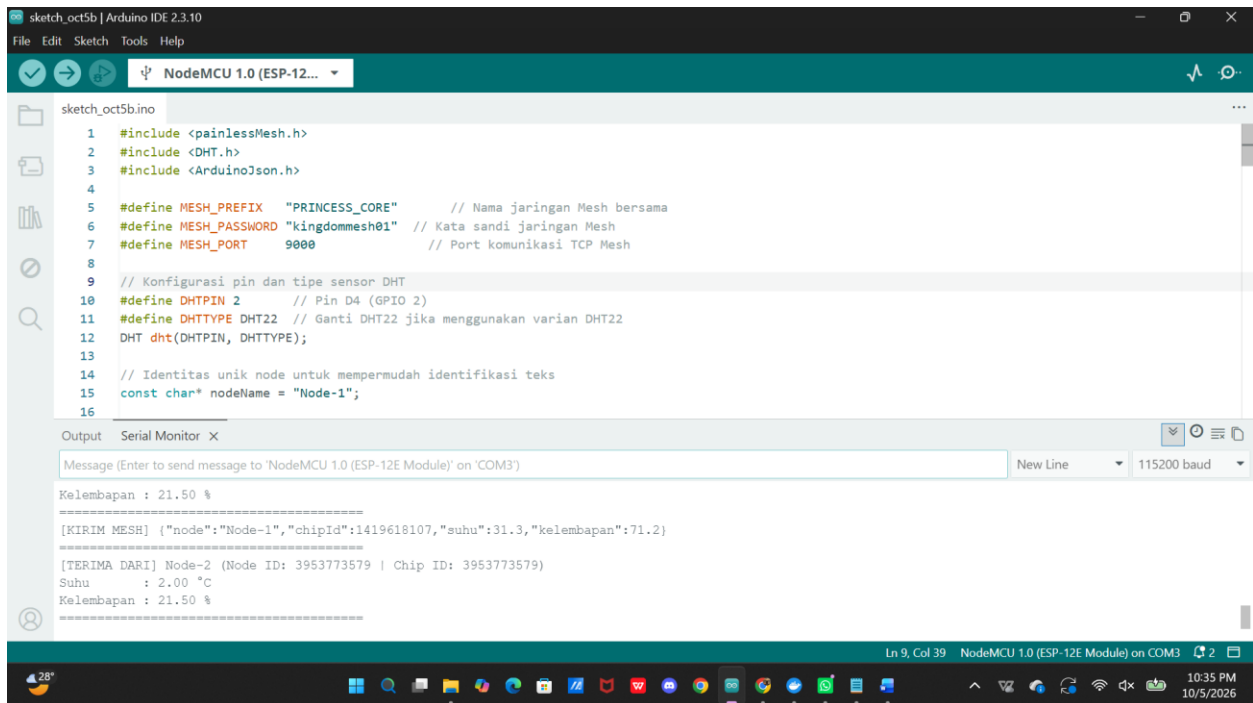
## Dokumentasi:



Pada Praktikum 5.5.3, diterapkan pola komunikasi telemetri *Many-to-One* menggunakan protokol ESP-NOW. Node 1 bertindak sebagai node sensor pengirim yang secara berkala mengirimkan paket data terstruktur (`struct_sensor`) berisi ID node, data simulasi suhu, dan kelembapan setiap 3 detik. Node 2 bertindak sebagai *Aggregator Hub / Base Station* yang menerima paket data tersebut lalu menyimpannya ke dalam array records berdasarkan ID pengirim sebelum mencetaknya ke Serial Monitor. Sistem berhasil melakukan koleksi data telemetri secara cepat tanpa bantuan router eksternal.

## 5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

### Node 1 (Pengirim):



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi jaringan Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 // Fungsi untuk mengirim pesan ke jaringan Mesh
18 void sendMessage() {
19     StaticJsonDocument<256> doc;
20     doc["node"] = nodeName;
21     doc["chipId"] = "1419618107";
22     doc["suhu"] = 31.3;
23     doc["kelembapan"] = 71.2;
24     String json = doc.toString();
25     mesh.send(json);
26 }
27
28 void setup() {
29     mesh.begin();
30     mesh.setMeshPrefix(MESH_PREFIX);
31     mesh.setMeshPassword(MESH_PASSWORD);
32     mesh.setMeshPort(MESH_PORT);
33     pinMode(DHTPIN, OUTPUT);
34 }
35
36 void loop() {
37     sendMessage();
38     delay(1000);
39 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

Kelembapan : 21.50 %

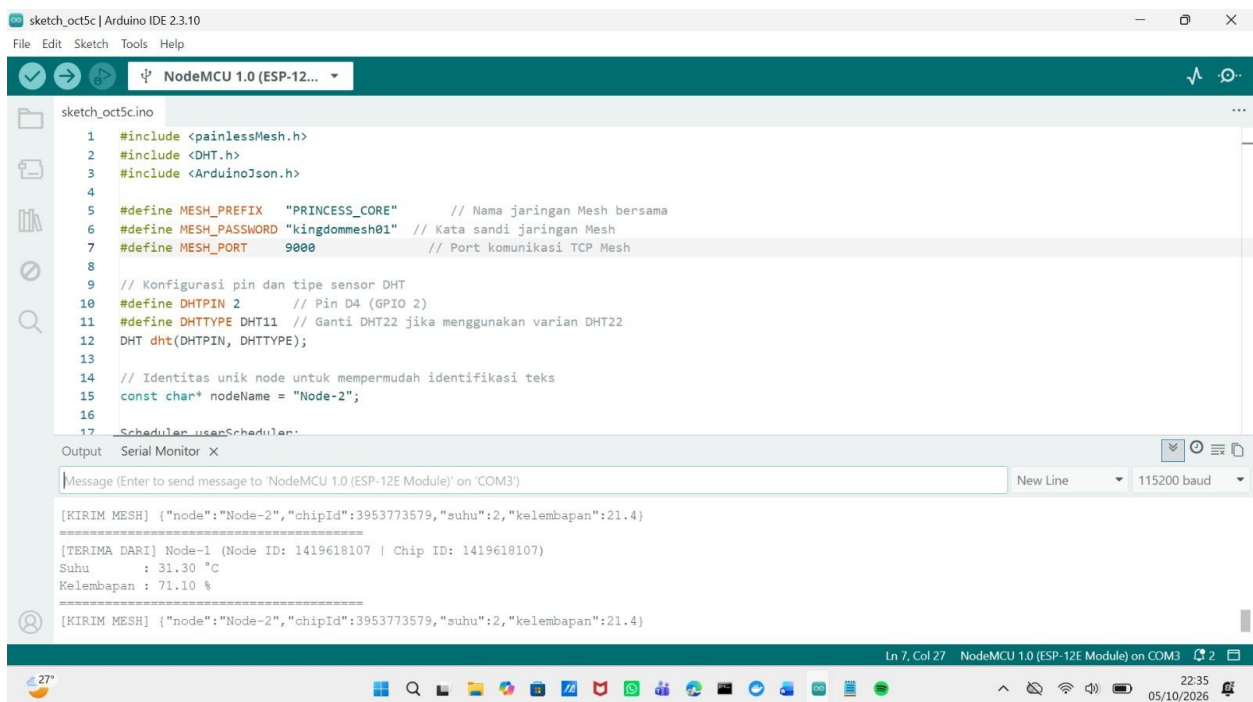
[KIRIM MESH] {"node":"Node-1","chipId":"1419618107","suhu":31.3,"kelembapan":71.2}

[TERIMA DARI] Node-2 (Node ID: 3953773579 | Chip ID: 3953773579)

Suhu : 2.00 °C

Kelembapan : 21.50 %

### Node 2 (Penerima):



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi jaringan Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT11 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-2";
16
17 // Fungsi untuk menerima pesan dari jaringan Mesh
18 void receiveMessage() {
19     StaticJsonDocument<256> doc;
20     String json;
21     while (mesh.receive(json)) {
22         doc.parse(json);
23         String node = doc["node"];
24         String chipId = doc["chipId"];
25         float suhu = doc["suhu"];
26         float kelembapan = doc["kelembapan"];
27         Serial.println("[KIRIM MESH] {"node\":\"Node-2\", \"chipId\":\"3953773579\", \"suhu\":2, \"kelembapan\":21.4}");
28     }
29 }
30
31 void setup() {
32     mesh.begin();
33     mesh.setMeshPrefix(MESH_PREFIX);
34     mesh.setMeshPassword(MESH_PASSWORD);
35     mesh.setMeshPort(MESH_PORT);
36     pinMode(DHTPIN, OUTPUT);
37 }
38
39 void loop() {
40     receiveMessage();
41     delay(1000);
42 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

[KIRIM MESH] {"node":"Node-2","chipId":"3953773579","suhu":2,"kelembapan":21.4}

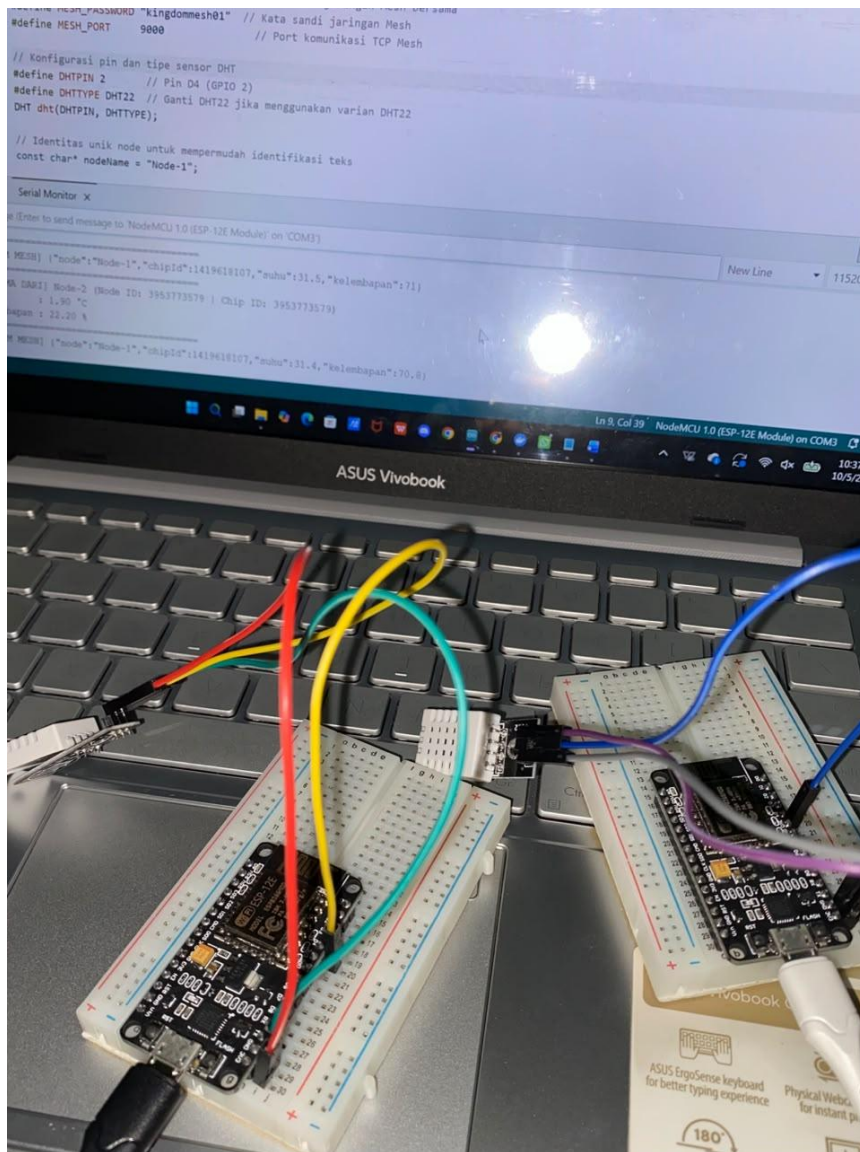
[TERIMA DARI] Node-1 (Node ID: 1419618107 | Chip ID: 1419618107)

Suhu : 31.30 °C

Kelembapan : 71.10 %

[KIRIM MESH] {"node":"Node-2","chipId":"3953773579","suhu":2,"kelembapan":21.4}

## Dokumentasi:



Pada Praktikum 5.5.4, dibangun jaringan sensor *Ad-Hoc Multi-Hop* otonom berbasis rantai *Mesh* menggunakan pustaka *painlessMesh*. Setiap perangkat (*Node-1* dan *Node-2*) saling terhubung membentuk topologi jaringan tanpa membutuhkan *router* pusat maupun pengetahuan *MAC Address* lawan secara manual.

Data telemetri disimulasikan dan dikemas dalam format JSON menggunakan pustaka *ArduinoJson*, kemudian disiarkan (*broadcast*) ke seluruh anggota jaringan setiap 3 detik lewat fungsi `mesh.sendBroadcast()`. Penggunaan `TaskScheduler` menggantikan `delay()` terbukti menjaga sinkronisasi perutean jaringan agar tidak terputus (*freeze*). Pada Serial Monitor (*115200 baud*), setiap *node* berhasil menerima, melakukan *parsing* JSON, dan menampilkan nilai suhu serta kelembapan dari *node* lain secara *real-time*.

- Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):**  
 Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (sendStatus == 0), lakukan eksperimen pemutusan simpul:
  - Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
  - Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim!  
Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
  - Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Node 1 (Pengirim): Saat USB dicolok

```

1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Andai
5 uint8_t receiver[] = {0x84, 0xF3, 0xEB, 0xA0, 0xCC, 0x00};
6
7 // Definisi struktur data paket instruksi
8 typedef struct struct_pesan {
9   int perintahId;
10  int nilaiParameter;
11 } struct_pesan;
12
13 struct_pesan paketKirim;
14
15 unsigned long previousMillis = 0;
16 const long interval = 2000; // Transmisi berkala setiap 2 detik

```

Serial Monitor X

```

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
Kirim paket ke: 84:f3:e8:a9:cc:00 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:e8:a9:cc:00 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:e8:a9:cc:00 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:e8:a9:cc:00 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:e8:a9:cc:00 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:e8:a9:cc:00 | Status: Berhasil Diterima
Kirim paket ke: 84:f3:e8:a9:cc:00 | Status: Berhasil Diterima

```

Node 2 (Penerima): Saat USB dicolok

```

1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6   int perintahId;
7   int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antena mendeteksi paket masuk
13 void onDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14   char macStr[18];
15   sprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16         mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17 }

```

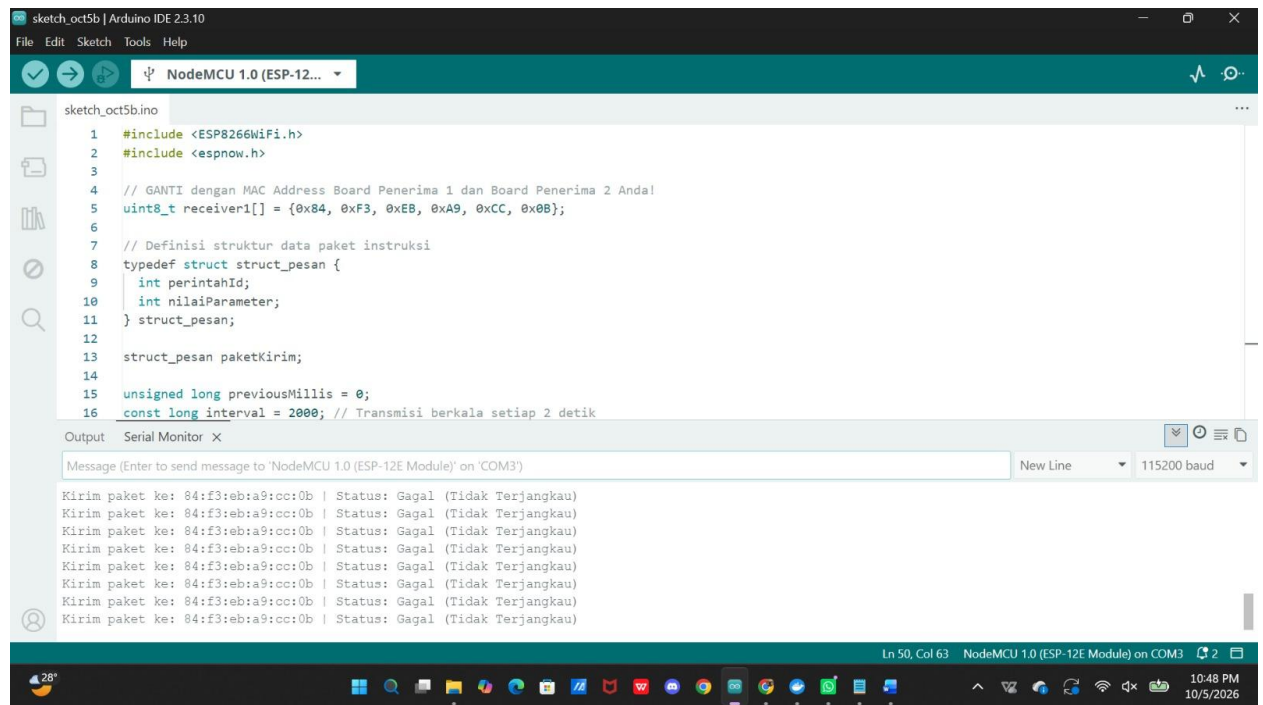
Serial Monitor X

```

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
Nilai Parameter: 53
-----
Paket masuk dari: 84:f3:e8:a9:cc:00
Ukuran Data: 0 byte
Perintah Id: 101
Nilai Parameter: 62
-----

```

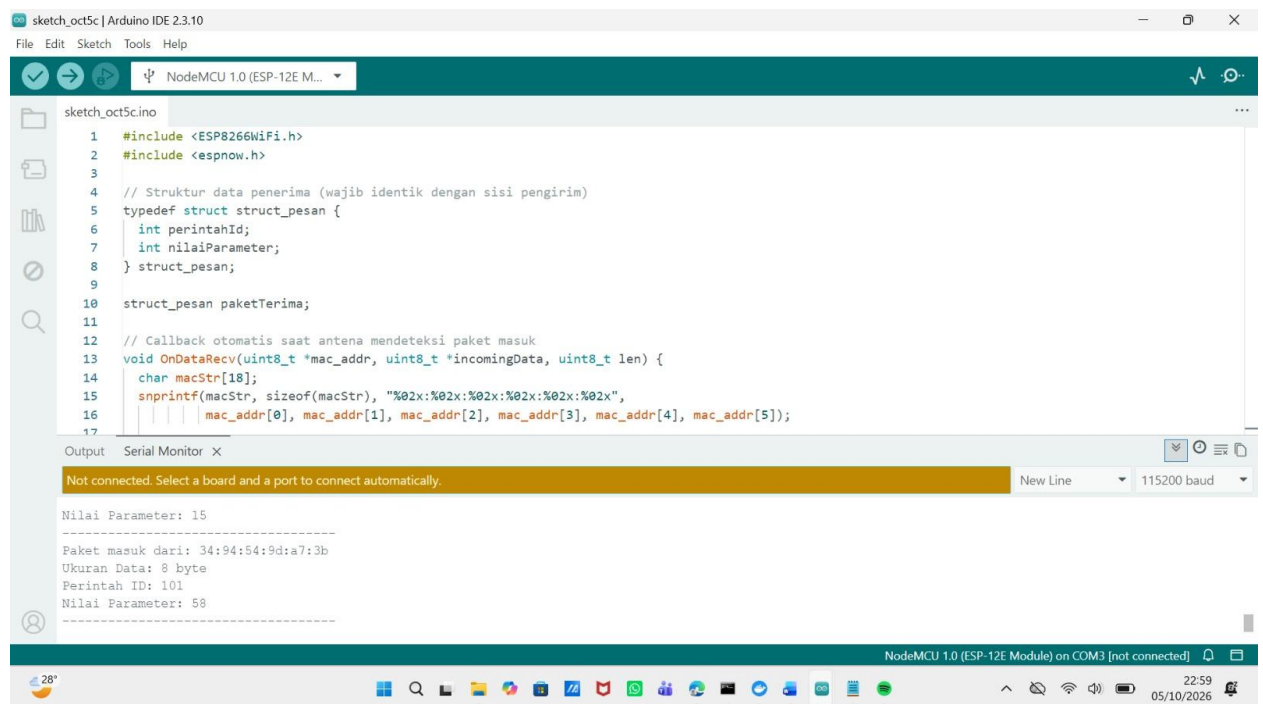
## Node 1 (Pengirim): Saat USB dicabut



The screenshot shows the Arduino IDE interface for Node 1 (Pengirim). The code in `sketch_oct5b.ino` includes `<ESP8266WiFi.h>` and `<espnow.h>`. It defines a MAC address `receiver1[]` and a packet structure `struct_pesanan` with fields `perintahId` and `nilaiParameter`. The code attempts to send a packet to the receiver. The serial monitor shows the following output:

```
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Gagal (Tidak Terjangkau)
Kirim paket ke: 84:f3:eb:a9:cc:0b | Status: Gagal (Tidak Terjangkau)
```

## Node 2 (Penerima): Saat USB dicabut



The screenshot shows the Arduino IDE interface for Node 2 (Penerima). The code in `sketch_oct5c.ino` includes `<ESP8266WiFi.h>` and `<espnow.h>`. It defines a packet structure `struct_pesanan` and a callback function `OnDataRecv` to handle incoming packets. The serial monitor shows the following output:

```
Not connected. Select a board and a port to connect automatically.

Nilai Parameter: 15
-----
Paket masuk dari: 34:94:54:9d:a7:3b
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 58
-----
```

**Hasil Observasi:** Ya, status pengiriman untuk MAC address milik Node 2 yang dicabut akan langsung berubah dari "Berhasil Diterima" menjadi "Gagal (Tidak Terjangkau)"

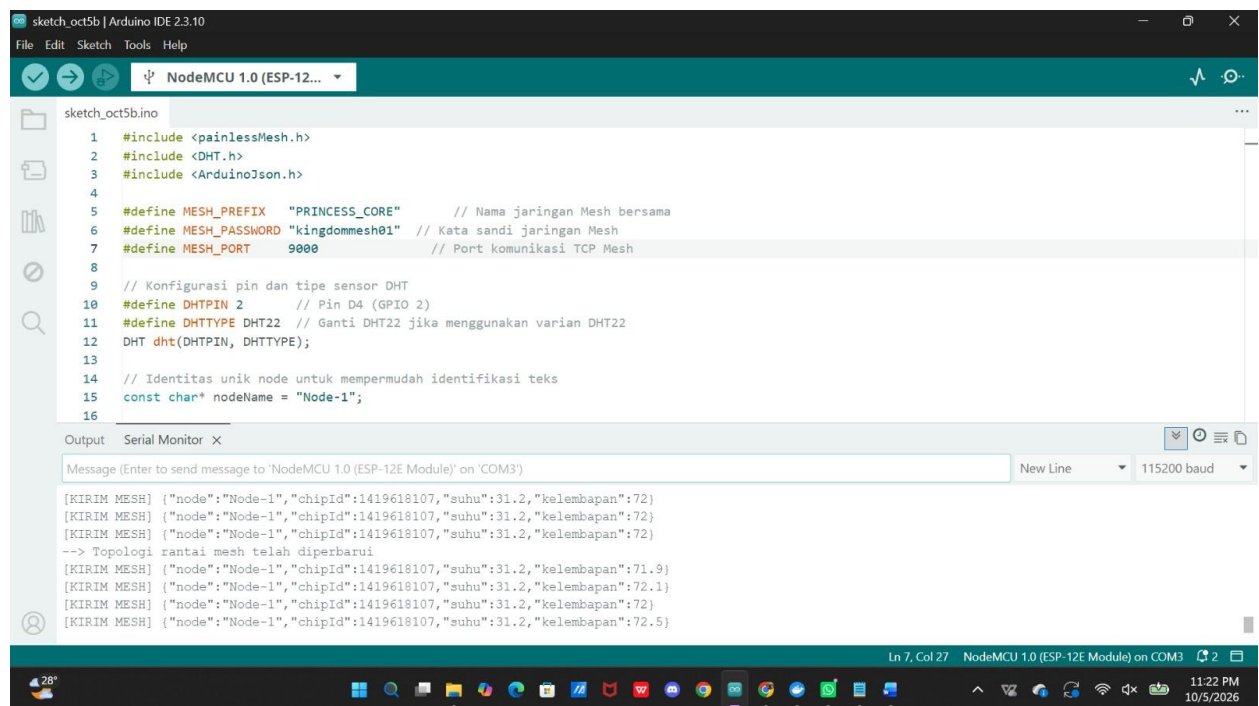
Protokol ESP-NOW bekerja pada **Data Link Layer (Layer 2 MAC)** menggunakan *802.11 Action Frames*. Masing-masing frame data yang dipancarkan oleh *Controller* mewajibkan adanya balasan berupa frame **ACK (Acknowledgment)** dari chip penerima fisik (*hardware-level ACK*). Ketika daya Node 2 dicabut, chip radio Node 2 berhenti merespons. Chip radio Node 1 yang tidak menerima sinyal ACK setelah batas waktu *retransmission* (timeout) internal habis akan menandai *ACK Failure* dan meneruskannya ke fungsi *callback* `OnDataSent()` dengan status kegagalan, tanpa membutuhkan protokol berbobot berat seperti TCP.

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):**

Jalankan jaringan *painlessMesh* pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID: ....

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?
- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Node 1 (Pengirim): Saat USB dicolok lalu dicabut



```
sketch_oct5b.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi jaringan Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

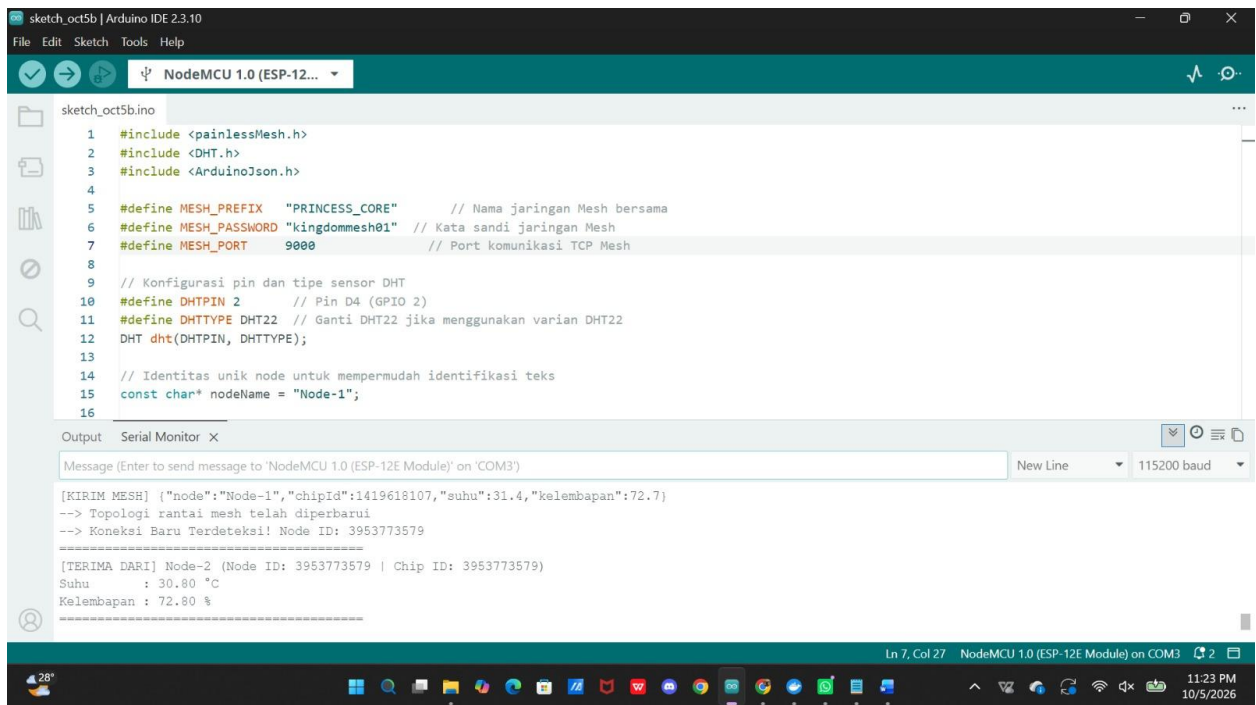
New Line 115200 baud

```
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72}
--> Topologi rantai mesh telah diperbarui
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":71.9}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72.1}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72}
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.2,"kelembapan":72.5}
```

Ln 7, Col 27 NodeMCU 1.0 (ESP-12E Module) on COM3 2

11:22 PM 10/5/2026

Node 1 (Pengirim): Saat USB dicolok lagi



The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for saving, compiling, uploading, and monitoring. The sketch is named 'sketch\_oct5b.ino' and is targeted to a 'NodeMCU 1.0 (ESP-12...)'. The code in the sketch includes headers for 'painlessMesh.h', 'DHT.h', and 'ArduinoJson.h'. It defines mesh parameters: 'PRINCESS\_CORE' as the prefix, 'kingdommesh01' as the password, and '9000' as the port. It also configures a DHT sensor on pin 2. The serial monitor shows the following output:

```
[KIRIM MESH] {"node": "Node-1", "chipId": "1419618107", "suhu": 31.4, "kelembapan": 72.7}
--> Topologi rantai mesh telah diperbarui
--> Koneksi Baru Terdeteksi! Node ID: 3953773579

[TERIMA DARI] Node-2 (Node ID: 3953773579 | Chip ID: 3953773579)
Suhu      : 30.80 °C
Kelembapan : 72.80 %
```

- **Waktu Deteksi Node Keluar:** Membutuhkan waktu sekitar **3–5 detik**.
  - *Penjelasan:* painlessMesh mengirimkan paket *keep-alive* / *heartbeat* secara periodik. Jika dalam interval tertentu paket balasan dari Node 2 hilang, topologi mengganggu node tersebut putus dan memanggil `changedConnectionCallback()`.
- **Waktu Re-join Otonom (Self-Healing):** Membutuhkan waktu sekitar **5–10 detik**.
  - *Penjelasan:* painlessMesh mengirimkan paket *keep-alive* / *heartbeat* secara periodik. Jika dalam interval tertentu paket balasan dari Node 2 hilang, topologi mengganggu node tersebut putus dan memanggil `changedConnectionCallback()`.
  - *Penjelasan:* Saat Node 2 menyala kembali, stack WiFi akan memindai *beacon* jaringan (PRINCESS\_CORE), melakukan *handshake* TCP pada port yang ditentukan, dan menyinkronkan jam jaringan secara otomatis tanpa butuh konfigurasi ulang.

- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):**

Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.

- i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- ii. Jika suhu bernilai normal ( $\leq 32.0$  °C), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetri dari Node <chipId> aman`.
- iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Uji Kondisi Normal

Node 1 (Pengirim):

```
1 #include <painlessMesh>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP
8
9 // Konfigurasi pin dan tipe sensor DHT22
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Menggunakan sensor DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node (Ubah "Node-1" pada board pertama, dan "Node-2" pada board kedua)
15 const char* nodeName = "Node-1";
16
```

Output Serial Monitor X

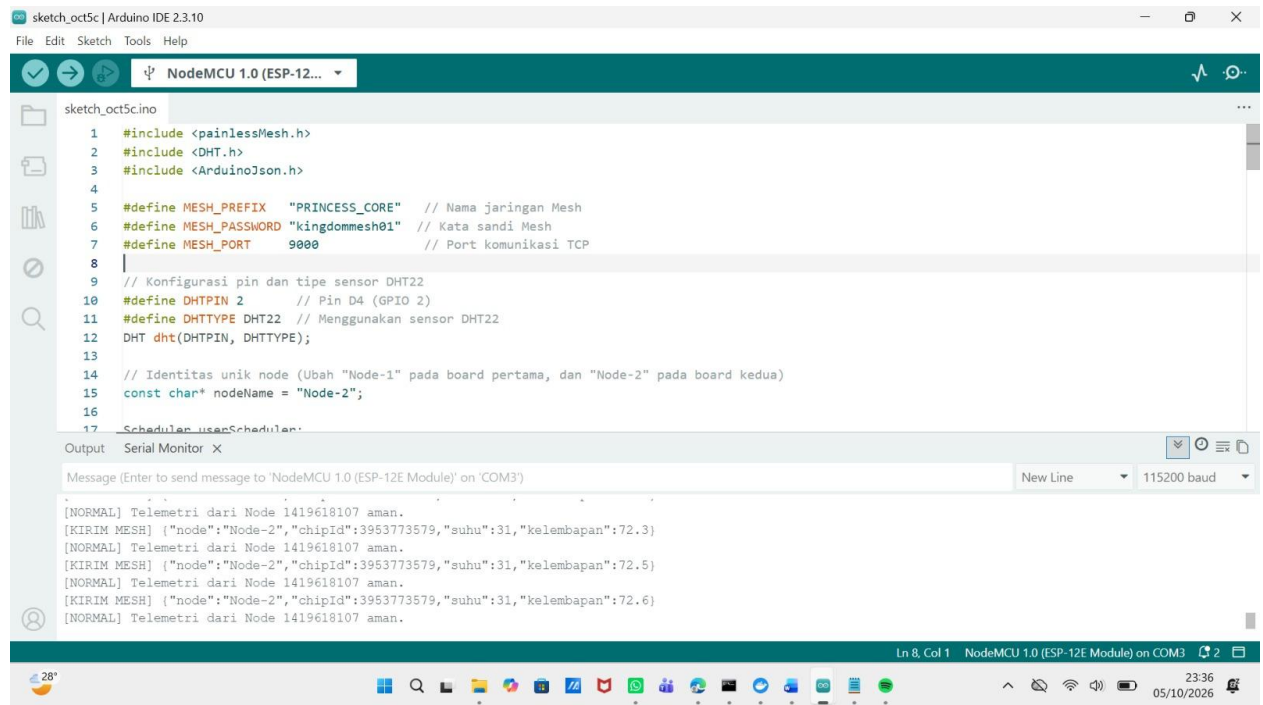
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[NORMAL] Telemetri dari Node 3953773579 aman.  
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31,"kelembapan":72.1}  
[NORMAL] Telemetri dari Node 3953773579 aman.  
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.1,"kelembapan":73.2}  
[NORMAL] Telemetri dari Node 3953773579 aman.  
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.1,"kelembapan":72.6}  
[NORMAL] Telemetri dari Node 3953773579 aman.  
[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":31.1,"kelembapan":72.3}

Ln 109, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM3 2

11:34 PM 10/5/2026

## Node 2 (Penerima):



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP
8
9 // Konfigurasi pin dan tipe sensor DHT22
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Menggunakan sensor DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node (Ubah "Node-1" pada board pertama, dan "Node-2" pada board kedua)
15 const char* nodeName = "Node-2";
16
17 #include <Scheduler.h>
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[NORMAL] Telemetri dari Node 1419618107 aman.

[KIRIM MESH] {"node":"Node-2","chipId":3953773579,"suhu":31,"kelembapan":72.3}

[NORMAL] Telemetri dari Node 1419618107 aman.

[KIRIM MESH] {"node":"Node-2","chipId":3953773579,"suhu":31,"kelembapan":72.5}

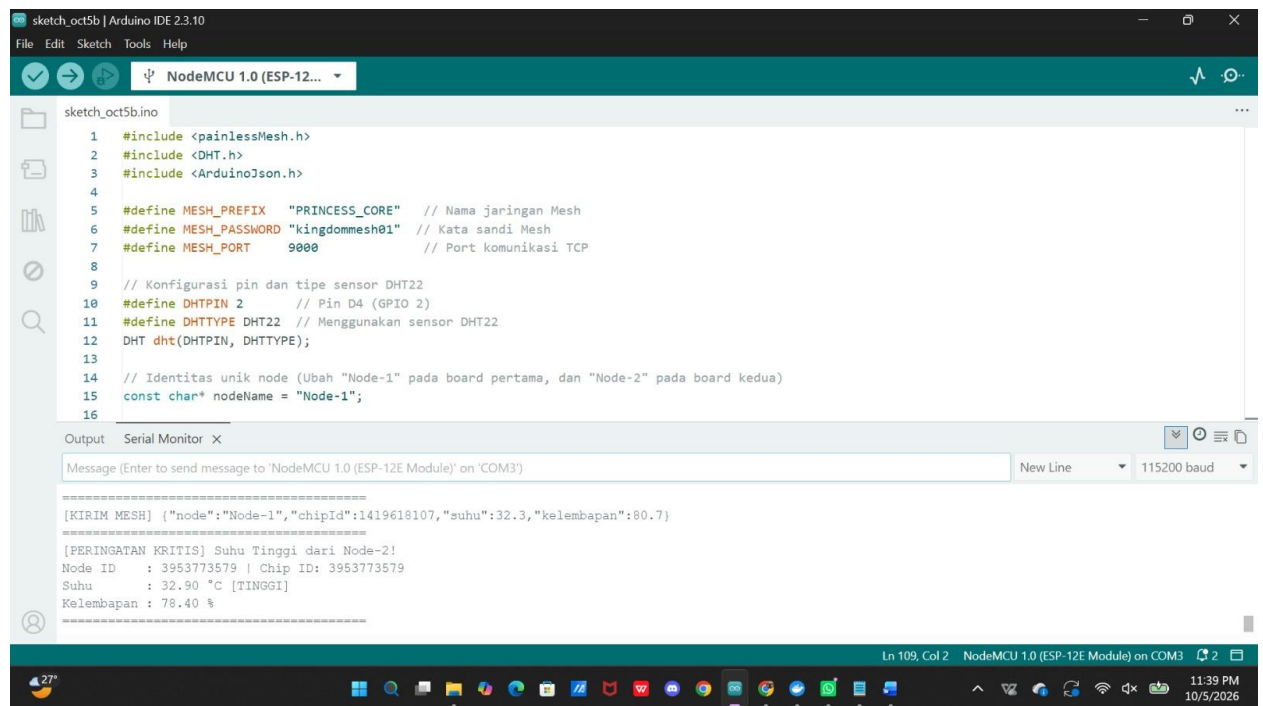
[NORMAL] Telemetri dari Node 1419618107 aman.

[KIRIM MESH] {"node":"Node-2","chipId":3953773579,"suhu":31,"kelembapan":72.6}

[NORMAL] Telemetri dari Node 1419618107 aman.

## Uji Kondisi Kritis

## Node 1 (Pengirim):



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP
8
9 // Konfigurasi pin dan tipe sensor DHT22
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Menggunakan sensor DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node (Ubah "Node-1" pada board pertama, dan "Node-2" pada board kedua)
15 const char* nodeName = "Node-1";
16
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[KIRIM MESH] {"node":"Node-1","chipId":1419618107,"suhu":32.3,"kelembapan":80.7}

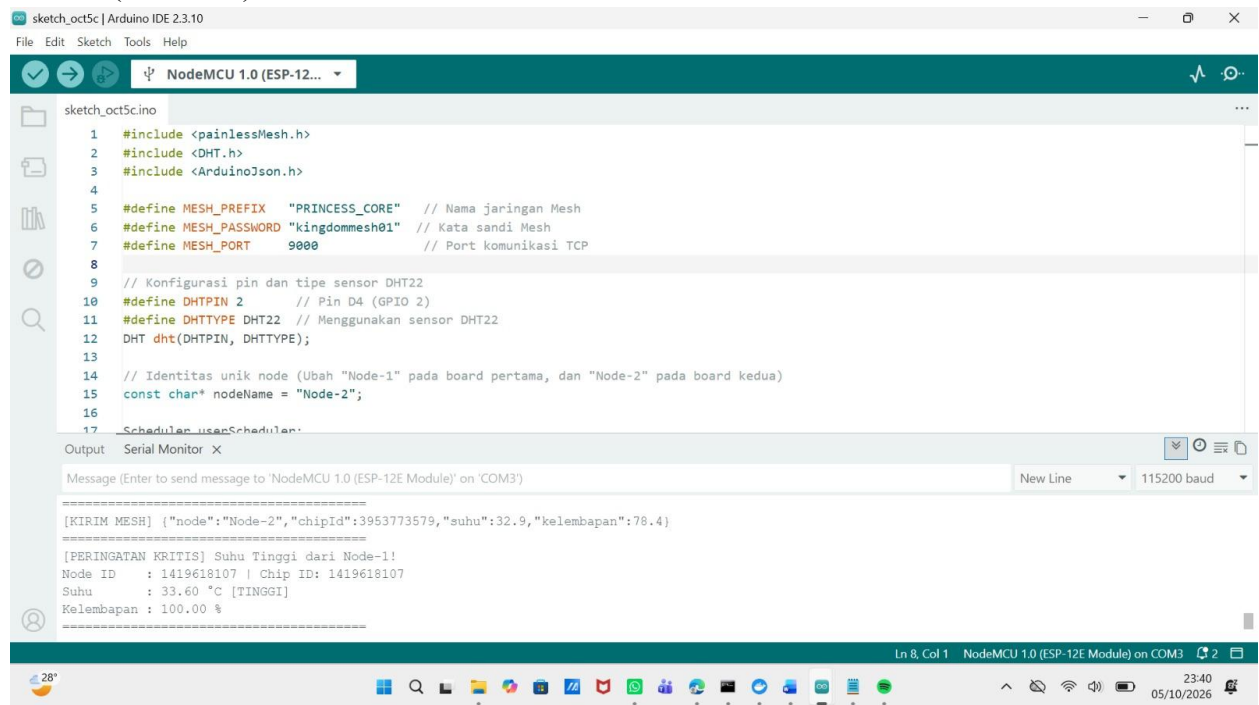
[PERINGATAN KRITIS] Suhu Tinggi dari Node-2!

Node ID : 3953773579 | Chip ID: 3953773579

Suhu : 32.90 °C [TINGGI]

Kelembapan : 78.40 %

## Node 2 (Penerima):



The screenshot shows the Arduino IDE interface with the sketch 'sketch\_oct5c.ino' open. The code is for a NodeMCU 1.0 (ESP-12E) module. It includes libraries for a custom mesh protocol, DHT sensor, and JSON parsing. The code defines mesh parameters like prefix, password, and port, and configures the DHT22 sensor. It also sets a unique node name 'Node-2'. The serial monitor shows the output of the code, including a message from Node-2 with temperature and humidity data, and a critical warning message from Node-1.

```
sketch_oct5c.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "PRINCESS_CORE" // Nama jaringan Mesh
6 #define MESH_PASSWORD "kingdommesh01" // Kata sandi Mesh
7 #define MESH_PORT 9000 // Port komunikasi TCP
8
9 // Konfigurasi pin dan tipe sensor DHT22
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Menggunakan sensor DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node (Ubah "Node-1" pada board pertama, dan "Node-2" pada board kedua)
15 const char* nodeName = "Node-2";
16
17 #include <Scheduler.h>
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[KIRIM MESH] {"node":"Node-2","chipId":3953773579,"suhu":32.9,"kelembapan":78.4}

[PERINGATAN KRITIS] Suhu Tinggi dari Node-1  
Node ID : 1419618107 | Chip ID: 1419618107  
Suhu : 33.60 °C [TINGGI]  
Kelembapan : 100.00 %

Blok fungsi receivedCallback(uint32\_t from, String &msg) hasil modifikasi:

```
void receivedCallback(uint32_t from, String &msg) {
    // Parsing muatan data JSON yang diterima dari jaringan mesh
    StaticJsonDocument<200> doc;
    DeserializationError error = deserializeJson(doc, msg);

    if (!error) {
        const char* sender = doc["node"];
        uint32_t chipId = doc["chipId"];
        float suhu = doc["suhu"];
        float kelembapan = doc["kelembapan"];

        // Filter ambang batas suhu kritis (> 32.0 °C)
        if (suhu > 32.0) {
            Serial.println("=====");
            Serial.printf("[PERINGATAN KRITIS] Suhu Tinggi dari %s!\n", sender);
            Serial.printf("Node ID : %u | Chip ID: %u\n", from, chipId);
            Serial.printf("Suhu : %.2f °C [TINGGI]\n", suhu);
        }
    }
}
```

```
Serial.printf("Kelembapan : %.2f %%\n", kelembapan);  
Serial.println("=====");  
} else {  
    // Tampilan status singkat jika suhu normal (<= 32.0 °C)  
    Serial.printf("[NORMAL] Telemetry dari Node %u aman.\n", chipId);  
}  
} else {  
    Serial.printf("[ERROR JSON DARI %u]: %s\n", from, msg.c_str());  
}  
}
```